Rapport Application mobile de gestion des commandes

1. Présentation et objectifs

Cette application mobile permet de gérer le cycle de vente d'une entreprise de distribution : suivre l'activité sur un tableau de bord, créer des commandes, gérer le catalogue de produits et consulter les clients. Elle reprend les fonctionnalités d'un site web PHP existant, qui continue de fonctionner : les deux partagent la même base de données et le même dossier de photos.

Les objectifs étaient les suivants :

- offrir une interface mobile simple, avec un onglet par grande fonction ;
- garantir la **cohérence du stock** à chaque commande ;
- protéger la base de données en n'exposant aucun identifiant MySQL dans l'application ;
- fournir des indicateurs fiables et rapides pour aider à la décision.

2. Architecture technique

Une application Flutter ne doit pas se connecter directement à MySQL : le mot de passe serait embarqué dans l'application et la base serait exposée. Le projet adopte donc une architecture en trois couches :

Couche	Technologie	Rôle
Présentation	Flutter (Dart, Material 3)	Écrans, navigation, saisie
Service	API PHP (PDO, JSON)	Logique métier, sécurité, transactions
Données	MySQL (InnoDB)	Stockage, vue et index

L'application envoie des requêtes HTTP à un point d'entrée unique (`index.php?r=<route>`). Chaque requête doit porter l'en-tête `X-API-KEY`, sans quoi l'API répond par une erreur 401. Le projet compte environ 2 200 lignes de Dart et une API de 10 routes.

Base de données

La base contient six tables : `cmd_departement`, `cmd_region`, `cmd_client`, `cmd_produit`, `cmd_commande` et `cmd_lignecommande`. Une commande appartient à un client, et chaque ligne associe un produit à une quantité (clé primaire composée commande + produit). Une vue, `cmd_v_ligne_detail`, calcule le montant de chaque ligne (quantité × prix) et sert de base à tous les indicateurs. Deux index (`idx_commande_date` et `idx_commande_date_client`) accélèrent les filtres par période.

Routes de l'API

Route	Méthode	Fonction
`dashboard`	GET	Indicateurs, graphiques, performance
`produits`, `produit`	GET	Liste avec recherche, fiche produit
`produit_enregistrer`	POST (multipart)	Ajout ou modification avec photo
`clients`	GET	Liste avec région, département, total
`commandes`, `commande`	GET	Liste et détail d'une commande
`commande_creer`	POST (JSON)	Nouvelle commande
`lignes_ajouter`	POST (JSON)	Ajout de plusieurs produits en transaction
`ligne_supprimer`	POST (JSON)	Retrait d'une ligne, stock rétabli

3. Fonctionnalités de l'application

L'application s'ouvre sur une barre de navigation à quatre onglets, un par page du site d'origine.

Tableau de bord. Il affiche cinq indicateurs sur une période choisie : chiffre d'affaires, nombre de commandes, nombre de clients, quantités vendues et panier moyen. Il présente aussi l'évolution mensuelle du chiffre d'affaires, le top 5 des produits et des clients, la répartition par département et la liste des produits à

réapprovisionner (stock inférieur ou égal à 10). Un écran d'analyse de performance affiche la durée de chaque requête SQL et le plan d'exécution (`EXPLAIN`).

Commandes. La liste est triée de la plus récente à la plus ancienne, avec le client et le montant. Une feuille « Nouvelle commande » permet de choisir un client et une date, puis l'utilisateur arrive dans le détail pour ajouter plusieurs produits en une seule validation ou retirer une ligne.

Produits. Le catalogue propose une recherche instantanée (avec temporisation de 400 ms pour limiter les requêtes), un formulaire de création et de modification et l'ajout d'une photo depuis la galerie ou l'appareil photo.

Clients. La liste indique pour chacun sa région, son département, son nombre de commandes et le montant total dépensé.

4. Réalisation côté Flutter

Le code est organisé pour rester lisible et maintenable :

- **`models/`** : une classe par type de données (`Produit`, `Client`, `Commande`, `Dashboard`) avec `fromJson`. Les écrans n'utilisent jamais de `Map` brute.
- **`services/api_service.dart`** : seul fichier qui connaît l'URL et le format JSON. Sa méthode `_traiter()` gère le délai d'attente de 15 secondes, le serveur injoignable, le JSON invalide et les codes d'erreur, et transforme tout en une `ApiException` au message clair en français.
- **`widgets/commun.dart`** : briques réutilisables (`Bloc`, `AsyncView`, `CarteKpi`, `ListeBarres`, `GraphiqueMensuel`, `Vignette`).
- **`screens/`** : un fichier par écran, tous construits sur le même schéma. `AsyncView` gère explicitement les trois états (chargement, erreur avec bouton « Réessayer », données), et tirer la liste vers le bas la rafraîchit.
- **`utils/format.dart`** : formatage des montants en FCFA et des dates en français, avec conversion tolérante des nombres que MySQL renvoie parfois sous forme de texte.

La langue est fixée en français (`flutter_localizations`), et le thème reprend la palette du site PHP. Les dépendances externes se limitent à `http` et `image_picker`.

5. Sécurité et fiabilité

Cohérence du stock. C'est le point critique. L'ajout de lignes s'exécute dans une transaction : chaque produit est verrouillé par `SELECT … FOR UPDATE`, dans un ordre constant (`ksort`) pour éviter les blocages entre utilisateurs. Si un seul produit manque de stock, tout est annulé (`rollBack`) et le message indique la quantité disponible. La suppression d'une ligne rétablit le stock dans une transaction également. Un même produit saisi deux fois est cumulé.

Sécurité de l'API.

- requêtes préparées PDO sans émulation, contre l'injection SQL ;
- clé API comparée avec `hash_equals` ;
- photos vérifiées par leur vrai type (`finfo`, JPG/PNG/WebP), limitées à 2 Mo, enregistrées sous un nom aléatoire ;
- erreurs techniques écrites dans les journaux serveur mais jamais renvoyées au client ;
- vérification de la méthode HTTP et validation de toutes les entrées (dates, nombres, longueurs).

Performance. Toutes les agrégations sont faites en SQL et jamais en Dart. La vue et les index limitent le coût des requêtes, les classements sont limités par `LIMIT`, et l'analyse de performance intégrée permet de vérifier l'usage des index.

6. Installation et tests

Le déploiement suit quatre étapes : importer les deux scripts SQL dans phpMyAdmin, copier le dossier `backend/` dans `htdocs` de XAMPP en changeant la clé `API_KEY`, créer le projet Flutter en y copiant `lib/` et `pubspec.yaml`, puis renseigner `config.dart`. L'adresse de l'API dépend de l'environnement : `10.0.2.2` pour l'émulateur

Android, l'adresse IP du PC pour un téléphone réel. Il faut aussi autoriser Internet et le trafic HTTP dans le manifeste Android.

Le scénario de test validé couvre : indicateurs variant selon les dates, recherche de produit, création d'une commande avec deux lignes (stock diminué), refus d'une quantité supérieure au stock sans aucune écriture, retrait d'une ligne (stock rétabli) et mise à jour des totaux clients.

7. Limites et perspectives

Les limites suivantes sont connues et assumées :

- le prix n'est pas figé à la vente, donc le chiffre d'affaires historique suit le prix actuel ;
- les listes ne sont pas paginées ;
- l'authentification repose sur une clé unique, sans compte utilisateur ;
- l'API fonctionne en HTTP, adapté au local mais pas à la production.

Les évolutions prioritaires seraient : enregistrer le prix de vente dans `cmd_lignecommande`, ajouter la pagination, mettre en place une connexion utilisateur avec jeton, passer en HTTPS, puis adopter un gestionnaire d'état (`provider` ou `riverpod`) si l'application grossit. La clé API par défaut (`CHANGER_MOI_cle_api_2026`) et les identifiants de base de données doivent être modifiés avant tout usage réel.

8. Conclusion

L'application répond au besoin : elle apporte sur mobile l'ensemble des fonctions du site PHP, avec une interface claire à quatre onglets, une API sécurisée et surtout une gestion rigoureuse du stock grâce aux transactions. La séparation modèles / service / écrans rend le code simple à faire évoluer. Les points restants (authentification, pagination, HTTPS, prix figé) constituent une feuille de route réaliste pour passer d'un prototype abouti à une application de production.